3 PROPOSTA DE SOLUÇÃO

O referencial teórico evidenciou que a portabilidade de *firmware* não depende apenas da existência de uma camada de abstração de *hardware*. Para que o reúso ocorra de forma sistemática, também é necessário impedir que tipos e convenções dos fabricantes alcancem a lógica comum, separar a configuração da placa das operações dos periféricos e definir contratos que possam ser implementados e verificados em plataformas distintas. As soluções analisadas atendem partes desse problema, mas deixam lacunas na combinação entre reúso da aplicação, reúso de componentes intermediários e reaproveitamento das adaptações de baixo nível entre projetos que utilizam a mesma plataforma.

Diante dessas lacunas, propõe-se um *framework* modular, escrito em C, baseado em uma arquitetura em camadas e destinado a microcontroladores de 32 *bits*. A expectativa técnica é que interfaces comuns de acesso ao *hardware*, combinadas com portes de plataforma e BSPs separados, permitam manter inalterados a aplicação e os componentes reutilizáveis quando o *firmware* for executado em microcontroladores de fabricantes distintos.

Este capítulo delimita o escopo da solução, deriva seus requisitos, descreve a arquitetura e os contratos previstos, define os artefatos a serem implementados e estabelece o procedimento de validação. A proposta não possui intuito de abstrair e implementar todos os periféricos e funcionalidades dos microcontroladores, limitando-se aos recursos necessários para implementação da prova de conceito.

3.1 DELIMITAÇÃO E REQUISITOS DA PROPOSTA

O recorte da proposta compreende quatro placas baseadas em microcontroladores de 32 *bits*: uma ESP32 DevKit V1 com ESP-WROOM-32 DevKit, uma Black Pill STM32F411CEU6, uma DevEBox com STM32F407VGT6 e uma Pro Micro com nRF52840. Apesar de haver quatro configurações de *hardware*, serão utilizados três ambientes de desenvolvimento: ESP-IDF, STM32CubeIDE e nRF Connect SDK. As duas placas STM32 foram incluídas deliberadamente para verificar se uma mesma implementação do porte baseado na STM32CubeF4 HAL pode ser reutilizada entre microcontroladores da família STM32F4, mantendo em BSPs, inicializações e arquivos de compilação as diferenças entre as placas (Espressif Systems, 2025; STMicroelectronics, 2026; Nordic Semiconductor, 2026).

As interfaces portáveis mínimas serão GPIO, I2C, UART e temporizador. A interface de temporização fornecerá o tempo decorrido em milissegundos a partir de um *timer* disponibilizado pela plataforma. Esse conjunto foi escolhido porque permite construir uma prova de conceito que reúne entrada digital, saída digital, dois dispositivos no mesmo barramento, comunicação serial e temporização, sem ampliar o trabalho para periféricos que não participam do

cenário avaliado. SPI, ADC, conectividade sem fio, armazenamento persistente, DMA avançado e abstrações de RTOS não integram o escopo. O *framework* também não se propõe a oferecer suporte universal a microcontroladores de 8, 16 e 32 *bits*; as conclusões deverão permanecer limitadas às quatro placas e aos três ambientes selecionados.

Para constituir uma linha de base, também serão desenvolvidas implementações equivalentes a prova de conceito sem a utilização do *framework* conceito, uma em cada ambiente selecionado, acopladas diretamente às APIs e aos tipos do ESP-IDF, da STM32CubeF4 HAL e do nRF Connect SDK. A implementação baseada na STM32CubeF4 HAL será construída para as duas placas STM32, resultando em quatro versões convencionais executáveis. Essas versões não utilizarão as interfaces, os portes, os BSPs nem os componentes do *framework*, mas poderão adotar a decomposição em funções e arquivos normalmente empregada em cada ambiente. Assim, a linha de base não será deliberadamente monolítica; sua característica distintiva será a ausência da camada comum proposta entre a aplicação e os SDKs.

O Quadro 2 apresenta os requisitos derivados das lacunas discutidas no referencial. Eles serão usados tanto para orientar a implementação futura quanto para impedir que decisões de baixo nível sejam acrescentadas sem relação com o problema investigado.

Quadro 2 – Requisitos da solução proposta

Código	**Requisito**
RF01	Manter a aplicação e os componentes comuns independentes de cabeçalhos, tipos e chamadas dos SDKs selecionados.
RF02	Confinar as diferenças de microcontrolador e de placa aos portes da HAL, aos BSPs, à inicialização e à configuração de compilação.
RF03	Permitir que um mesmo porte da HAL seja empregado por diferentes aplicações que utilizem a mesma plataforma, sem incorporar política de produto.
RF04	Definir contratos observáveis para GPIO, I2C, UART e temporizador em milissegundos, incluindo erros, *timeout*, bloqueio, inicialização, propriedade de recursos, concorrência e operações não suportadas.
RF05	Manter tarefas, filas, semáforos, escalonamento e demais primitivas de RTOS fora do contrato central do *framework*.
RF06	Fornecer componentes reutilizáveis que recebam suas dependências por interfaces comuns e não configurem diretamente o *hardware*.
RF07	Identificar a versão dos contratos implementados por cada porte e verificar sua conformidade antes da integração.
RF08	Executar a mesma prova de conceito nas quatro placas, preservando os arquivos-fonte comuns e o comportamento externo definido.
RF09	Reutilizar sem alteração o mesmo porte STM32CubeF4 nas construções para STM32F411CEU6 e STM32F407VGT6, mantendo as diferenças de placa fora desse porte.
RF10	Implementar versões equivalentes sem utilização do *framework* , acopladas diretamente aos três SDKs, para comparar de forma pareada o código compartilhado, o código mantido, a dispersão das dependências plataforma e o uso de memória.

Fonte: O Autor (2026).

A proposta resultará em um conjunto de artefatos, e não apenas em um diagrama arquitetural. Estão previstos os cabeçalhos das interfaces comuns, as implementações dessas interfaces para cada plataforma, os BSPs, os componentes reutilizáveis, uma aplicação demonstradora

e os projetos sem a utilziação do *framework*. A separação desses artefatos permite avaliar individualmente a estabilidade das interfaces, o isolamento do código específico, a preservação da lógica comum e as diferenças entre as duas formas de implementação.

3.2 ARQUITETURA GERAL E REGRA DE DEPENDÊNCIAS

A arquitetura proposta seguirá a divisão de responsabilidades apresentada na Figura 4, na qual a camada de aplicação reunirá as tarefas responsáveis pela execução da prova de conceito. Cada tarefa coordenará os componentes relacionados à sua responsabilidade, enquanto o acesso aos periféricos permanecerá encapsulado nos componentes reutilizáveis de botão, LED, SHT31-D, SSD1306, telemetria e temporização. Desse modo, nem as tarefas da aplicação nem esses componentes acessarão diretamente BSPs ou serviços dos fabricantes, pois toda interação com o *hardware* ocorrerá por meio das interfaces da HAL.

A tradução dos contratos comuns para cada ambiente de desenvolvimento ficará a cargo dos portes da HAL, ao passo que o BSP identificará pinos, instâncias de barramentos, periféricos e os demais recursos concretos de cada placa. Como essas responsabilidades permanecerão separadas, placas atendidas pelo mesmo ambiente e por contratos compatíveis poderão selecionar um único porte, ainda que necessitem de BSPs distintos.

Figura 4 – Arquitetura do *framework* proposto



Fonte: elaborada pelo autor.

A direção das dependências será preservada também nas inclusões de cabeçalhos. Os arquivos da aplicação dependerão somente das interfaces públicas da própria aplicação e dos componentes reutilizáveis, enquanto estes poderão utilizar interfaces de outros componentes e

os contratos comuns da HAL, sem incluir cabeçalhos de portes, BSPs ou SDKs. As interfaces da HAL permanecerão independentes de suas implementações concretas, cabendo a cada porte incluir apenas os contratos que implementa e os cabeçalhos do respectivo SDK. De forma semelhante, o BSP poderá utilizar as interfaces da HAL e o SDK para descrever e inicializar os recursos da placa, mas não conhecerá módulos da aplicação ou dos componentes.

Embora portes e BSPs ocupem a mesma região inferior da arquitetura, suas responsabilidades não se confundem: a HAL e seus portes expressarão operações portáveis, como escrever em um GPIO ou realizar uma transferência I2C, enquanto o BSP indicará quais instâncias concretas representam o botão, o LED, o sensor, o display e a UART de telemetria em determinada placa. Permanecerão no BSP e nos elementos de integração as decisões sobre *clocks*, pinos, barramentos e demais particularidades da compilação, de modo que a proposta não elimine as diferenças entre plataformas, mas determine onde elas deverão residir.

Nas duas placas STM32, essa separação será exercitada pela utilização dos mesmos arquivos-fonte do porte STM32F4 em ambas as construções, combinados com um BSP, um código de inicialização e uma configuração de compilação próprios para cada placa. Pinos, instâncias de periféricos, *clocks* e demais características dos microcontroladores deverão permanecer nesses elementos específicos; caso a integração exija modificar o porte ou introduzir nele condicionais destinadas a apenas um dos dispositivos, a alteração será tratada como evidência de insuficiência da fronteira proposta.

A divisão da aplicação em tarefas será mantida independente da presença de um RTOS, pois, no código comum, cada tarefa corresponderá a um módulo com contexto próprio e operações de inicialização e processamento. A forma de executar essas operações ficará no código específico de cada plataforma: nas configurações *bare-metal*, elas poderão ser chamadas pelo laço principal, enquanto no nRF Connect SDK poderão ser associadas a contextos fornecidos pelo Zephyr. Essa variação permanecerá fora do núcleo do *framework*, que não criará uma camada de abstração para sistema operacional e nem buscará uniformizar tarefas, filas, mutexes ou escalonadores.

Como os módulos comuns não conhecerão o BSP, os portes ou os SDKs, a ligação entre essas partes ficará concentrada no código de integração específico de cada placa. Esse código reunirá os arquivos comuns, o porte da HAL, o BSP, a inicialização e a configuração de compilação necessárias à geração do *firmware*. Seu ponto de entrada, representado, por exemplo, por um arquivo `main.c`, funcionará como raiz de composição ao obter os recursos concretos da placa, criar os contextos das tarefas e dos componentes, fornecer suas dependências e iniciar o processamento da aplicação.

Para realizar essa montagem, a raiz de composição poderá conhecer simultaneamente as interfaces públicas da aplicação e dos componentes, as interfaces e o porte selecionado da HAL, o BSP e os elementos necessários do SDK. Esse conhecimento ficará restrito à integração do executável, sem incorporar políticas que são responsabilidades da aplicação. A composição,

portanto, não constitui uma camada adicional da arquitetura mostrada na Figura 4, mas o ponto em que as implementações concretas serão conectadas ao código comum sem criar dependência direta entre a aplicação e o BSP.

As implementações sem a utilização do *framework* permanecerão fora da arquitetura mostrada na Figura 4. Em cada uma delas, a aplicação e os módulos funcionais poderão utilizar diretamente os recursos do respectivo SDK, sem reproduzir sob outros nomes os contratos, os portes ou os BSPs propostos. Essa delimitação evita atribuir à linha de base os mecanismos cuja contribuição se pretende avaliar, mas não impede o uso de funções, arquivos separados e recursos fornecidos pelo próprio ambiente de desenvolvimento.

3.3 INTERFACES E CONTRATOS PORTÁVEIS

As interfaces da HAL serão definidas em cabeçalhos públicos escritos em C. Os recursos concretos serão representados por *handles* opacos, evitando que estruturas ou tipos dos fabricantes sejam expostos aos componentes. A inicialização de baixo nível ficará sob responsabilidade do BSP e do código de composição; os componentes receberão referências já válidas para os recursos que utilizam e não assumirão sua propriedade.

Todas as operações retornarão códigos de estado pertencentes ao contrato comum. O conjunto mínimo distinguirá sucesso, erro geral, *timeout*, argumento inválido e operação não suportada. Operações de I2C e UART receberão um limite de tempo explícito e terão comportamento síncrono no recorte inicial. Chamadas a partir de rotinas de interrupção e uso concorrente do mesmo *handle* não serão garantidos pela interface comum; quando necessários, deverão ser tratados pela integração específica da aplicação. Essa restrição evita prometer reentrância ou segurança entre tarefas sem que a semântica possa ser demonstrada de forma equivalente nos três ambientes.

O Quadro 3 relaciona cada interface às operações necessárias para a prova de conceito e aos aspectos que deverão ser especificados antes da implementação.

Quadro 3 – Interfaces portáveis previstas

Interface	**Operações mínimas**	**Elementos do contrato**
`hal_gpio`	Leitura e escrita de nível lógico	Estado de inicialização, polaridade definida pelo BSP, argumentos inválidos e propriedade do recurso.
`hal_i2c`	Escrita, leitura e transação combinada	Endereço do dispositivo, buffers, comprimento, *timeout*, erros de barramento e serialização do acesso compartilhado.
`hal_uart`	Transmissão de bloco de dados	Instância utilizada, validade do *buffer*, tamanho do buffer, comportamento bloqueante, *timeout* e transmissão parcial.
`hal_timer`	Leitura do tempo decorrido em milissegundos	Fonte de temporização da plataforma, resolução de 1 ms.

Fonte: O Autor (2026).

O compartilhamento do barramento I2C entre o SHT31-D e o display exigirá que ambos recebam a mesma instância de `hal_i2c`, mantendo endereços e estado próprios em seus contextos. A aplicação será responsável por não iniciar operações concorrentes sobre o mesmo barramento. Essa decisão mantém o mecanismo de comunicação reutilizável sem impor mutexes ou uma política de escalonamento ao *framework*.

3.4 ORGANIZAÇÃO DOS ARTEFATOS

Os artefatos serão organizados de acordo com sua estabilidade e responsabilidade. O Quadro 4 apresenta a estrutura lógica prevista dos arquivos. Os nomes indicam fronteiras de responsabilidade, mas não fixam antecipadamente todos os detalhes do sistema de compilação, que deverão acompanhar as ferramentas de cada plataforma.

Quadro 4 – Artefatos previstos para o *framework*

Grupo	**Conteúdo e responsabilidade**
`hal/interface`	Cabeçalhos públicos, códigos de estado, tipos opacos, versões e documentação dos contratos.
`hal/ports`	Tradução das interfaces para ESP-IDF, STM32CubeF4 HAL, nRF Connect SDK.
`boards`	Mapeamento de pinos, instâncias de periféricos, endereços dos dispositivos e composição específica de cada placa.
`components`	Componentes de botão, LED, SHT31-D, SSD1306 e telemetria, dependentes apenas das interfaces comuns.
`app`	Estado e fluxo da aplicação, seleção da tela e tratamento dos eventos produzidos pelos componentes.
`docs`	Contratos, capacidades suportadas, versões.

Fonte: O Autor (2026).

Os componentes não serão apenas arquivos de implementação. Cada um deverá possuir interface pública, contexto próprio, parâmetros de inicialização e documentação de dependências. O componente do SHT31-D, por exemplo, receberá uma referência para I2C e o endereço do dispositivo; o componente de *display* receberá a mesma interface de barramento e manterá seu próprio contexto.

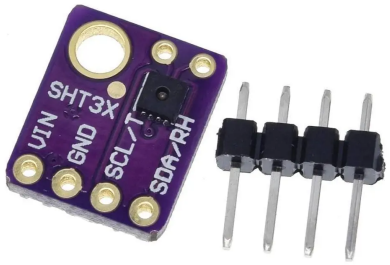
Além dos diretórios de primeiro nível apresentados na Quadro 4, a estrutura interna seguirá a mesma separação de responsabilidades. Em `hal/interface`, cada contrato portável será mantido em um subdiretório próprio, como GPIO, I2C, UART e temporizador. Em `hal/ports`, haverá um subdiretório para cada porte selecionado, e cada porte conterá sua própria divisão por periférico implementado. O mesmo critério será aplicado aos demais grupos: placas, componentes, tarefas da aplicação, testes e documentação deverão ser separados por unidade de responsabilidade, evitando que arquivos de naturezas distintas sejam agrupados apenas por conveniência.

3.5 PROVA DE CONCEITO PROPOSTA

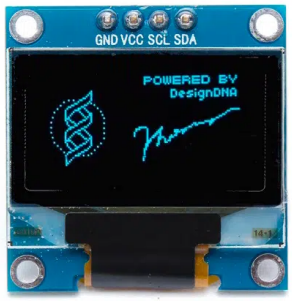
A prova de conceito será uma aplicação que irá medir temperatura e umidade do ambiente, exibindo as informações em um *display*. Um sensor SHT31-D fornecerá temperatura e umidade por I2C, enquanto um *display Organic Light-Emitting Diode* (OLED) de 0,96 polegada, resolução de 128 por 64 pontos, controlador SSD1306 e interface I2C apresentará as informações ao usuário. O sensor e o controlador selecionados oferecem as interfaces necessárias a essa composição (Sensirion, 2026; Solomon Systech, 2026).

Os módulos de sensor e *display* previstos são apresentados na Figura 5. Ambos compartilharão o mesmo barramento I2C, o que permitirá exercitar a composição de mais de um componente sobre uma única interface portável.

Figura 5 – Dispositivos I2C previstos para a prova de conceito



(a) Módulo sensor SHT31-D



(b) Módulo de display OLED SSD1306

Fonte: (a) Mercado Livre (s.d.c); (b) MakerHero (s.d.).

Os dispositivos externos de GPIO previstos são apresentados na Figura 6. O botão fornecerá a entrada digital usada para alternância das telas, enquanto o LED será empregado como indicação visual do estado da aquisição.

Figura 6 – Dispositivos GPIO previstos para a prova de conceito



(a) Botão tátil momentâneo de 6 x 6 mm



(b) LED amarelo difuso de 5 mm

Fonte: (a) Mercado Livre (s.d.a); (b) Mercado Livre (s.d.b).

Para realizar as operações de entrada e saída digital, a montagem utilizará também um botão tátil momentâneo, normalmente aberto, e um LED, ambos externos às placas. Serão empregados os mesmos componentes e circuitos nas quatro configurações microcontroladores,

mesmo quando a placa disponibilizar algum recurso embarcado equivalente, evitando que diferenças de polaridade, ligação elétrica ou finalidade desses recursos alterem a prova de conceito. O botão será conectado entre o GPIO e o terra, com um resistor externo de $10\,\mathrm{k}\Omega$ entre a entrada e $3.3\,\mathrm{V}$, resultando em acionamento ativo em nível baixo. O LED será conectado a uma saída GPIO por meio de um resistor em série de $1\,\mathrm{k}\Omega$, com acionamento ativo em nível alto. Os pinos e as polaridades serão descritos pelo BSP de cada placa, sem alterar os componentes reutilizáveis.

A aplicação realizará leituras periódicas do sensor em um intervalo configurado previamente. A primeira tela mostrará os valores atuais de temperatura e umidade. A segunda apresentará informações de diagnóstico reduzidas ao estado da última aquisição e ao tempo de operação. Um único acionamento do botão alternará entre as duas telas, com *debounce* de aproximadamente $30\,\mathrm{ms}$ realizado por software; não haverá pressionamento longo, menus, alteração de parâmetros ou persistência de configuração. O LED permanecerá aceso durante a operação normal, piscará quando houver falha de comunicação e ficará apagado antes da conclusão da inicialização. A UART transmitirá as leituras e os códigos de erro em um formato comum às quatro placas.

Quando uma operação I2C não for concluída, a aplicação preservará o último valor válido, apresentará o estado de erro, atualizará o LED e registrará a ocorrência pela UART. A política de repetição permanecerá na aplicação, enquanto o componente do sensor apenas devolverá o resultado da operação. Essa separação permite verificar se os contratos de erro são mantidos pelos três portes de plataforma e pelas quatro configurações de placa sem transferir decisões do produto para a biblioteca reutilizável.

A temporização será não bloqueante. Em cada plataforma, um *timer* fornecerá à interface comum a contagem do tempo decorrido em milissegundos. A aplicação utilizará essa contagem para determinar o instante da próxima aquisição, realizar o *debounce* do botão e controlar a indicação visual de falha. A aplicação comum será processada por chamadas sucessivas, sem criar tarefas próprias. Com isso, o mesmo fluxo poderá ser integrado ao laço principal ou a uma tarefa fornecida pelo ambiente de plataforma sem que componentes intermediários passem a depender de uma API de RTOS.

O Quadro 5 resume as quatro configurações previstas. Em todas elas serão mantidos o mesmo sensor, *display*, botão, LED, circuitos externos e comportamento da aplicação. Nas mudanças entre ambientes de desenvolvimento serão substituídos o porte, o BSP, a inicialização e o sistema de compilação. Entre as duas placas STM32, o porte deverá permanecer inalterado, e somente os demais elementos específicos da placa serão substituídos.

Cada placa receberá duas versões funcionalmente equivalentes do *firmware*: uma construída com o *framework* e outra derivada da implementação convencional correspondente ao seu ambiente. As duas versões utilizarão os mesmos dispositivos externos, mapeamento de pinos, intervalo de aquisição, formato de telemetria, políticas de erro e cenários de teste.

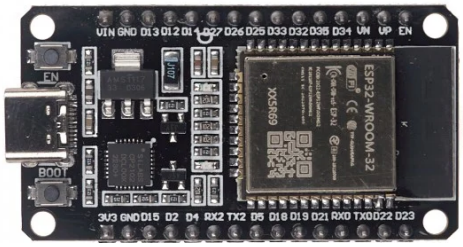
Quadro 5 – Plataformas previstas para a prova de conceito

Placa	Microcontrolador	Ambiente e adaptação
ESP32 ESP-WROOM-32 DevKit V1	ESP32	ESP-IDF e porte específico da HAL.
Black Pill STM32F411CEU6	STM32F411CEU6	STM32CubeF4 HAL, porte STM32F4 compartilhado e BSP próprio.
DevEBox STM32F407VGT6	STM32F407VGT6	STM32CubeF4 HAL, mesmo porte da STM32F411CEU6 e BSP próprio.
Pro Micro nRF52840	nRF52840	nRF Connect SDK, configuração da placa e porte específico da HAL.

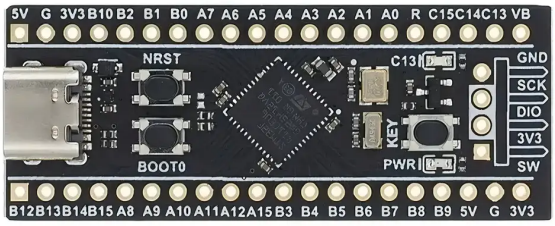
Fonte: O Autor (2026).

As placas de desenvolvimento selecionadas são apresentadas na Figura 7. A identificação visual complementa o Quadro 5 e delimita as configurações físicas que serão utilizadas na implementação futura.

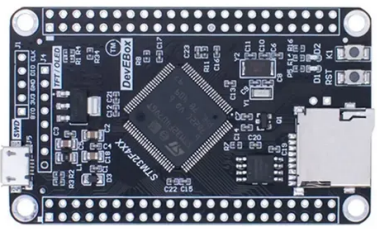
Figura 7 – Placas de desenvolvimento previstas para a prova de conceito



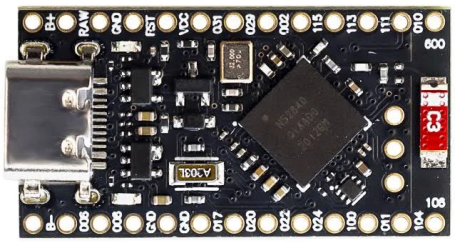
(a) ESP32 ESP-WROOM-32 DevKit V1



(b) Black Pill STM32F411CEU6



(c) DevEBox STM32F407VGT6



(d) Pro Micro nRF52840

Fonte: (a) Voltaat (s.d.); (b) Mercado Livre (s.d.e); (c) Mercado Livre (s.d.d); (d) Elecrow (s.d.).

3.6 PLANO DE VALIDAÇÃO

A proposta será considerada funcionalmente demonstrada somente se os mesmos cenários forem executados nas duas versões de cada uma das quatro placas: inicialização e aquisição válida; apresentação das duas telas; alternância pelo botão; transmissão pela UART; repetição periódica; e tratamento observável de falha na comunicação com o sensor. Os valores medidos poderão variar com o ambiente, mas a sequência de estados e as respostas a cada evento deverão permanecer equivalentes.

A evidência estrutural verificará se a portabilidade foi obtida pela arquitetura proposta, e não por cópias adaptadas da aplicação. Para isso, os arquivos da aplicação, dos componentes e das interfaces comuns deverão ser iguais entre as quatro construções baseadas no *framework*. Também será verificada a ausência de cabeçalhos e símbolos dos fabricantes nesses arquivos. Cada porte deverá executar os mesmos contratos e declarar explicitamente as capacidades que oferece. Adicionalmente, as versões STM32F411CEU6 e STM32F407VGT6 deverão referenciar exatamente os mesmos arquivos do porte STM32CubeF4, podendo apenas alterar as *macros* de configuração do SDK do fabricante.

O percentual de código compartilhado será calculado para cada placa, separadamente nas versões baseadas no *framework* e nas implementações convencionais, conforme a Equação 3.1. Serão consideradas apenas linhas de código-fonte e cabeçalhos desenvolvidos no trabalho. Código dos fabricantes, arquivos gerados, produtos de compilação e testes não participarão do cálculo.

$$P_{\text{comp}} = \frac{L_{\text{comum}}}{L_{\text{comum}} + L_{\text{específico}}} \times 100 \tag{3.1}$$

Nessa expressão, L_{comum} corresponde às linhas dos mesmos arquivos que são incluídos, sem alteração e sem condicionais de compilação específicas de placa ou SDK, nas quatro construções da forma de implementação avaliada. Por sua vez, $L_{\text{específico}}$ corresponde às linhas necessárias à placa considerada que não integram esse conjunto comum, incluindo porte, BSP, inicialização e integração. Cada placa e forma de implementação terão o percentual de código compartilhado calculados individualmente.

Assim, a métrica indicará qual parcela do código de cada versão provém de artefatos comuns mantidos em uma única origem. Ela não será interpretada isoladamente como medida de qualidade ou de esforço de desenvolvimento, pois um percentual elevado também poderia resultar de interfaces excessivamente limitadas. Sua análise será combinada com os cenários funcionais, o volume total de código mantido e a dispersão das dependências de plataforma.

Para esse cálculo, o porte STM32CubeF4 permanecerá classificado como código específico do ambiente STM32, pois não é compartilhado com ESP32 e nRF52840. Seu reaproveitamento entre as duas placas STM32 será contabilizado separadamente pela identidade dos arquivos e de seus hashes. Essa separação evita somar o mesmo reúso simultaneamente ao percentual global e à evidência intrafamília.

A conformidade dos portes será verificada por um roteiro comum associado aos contratos da HAL. Esse roteiro não terá como objetivo demonstrar cobertura exaustiva dos periféricos, mas confirmar que cada porte implementa o mesmo comportamento observável exigido pela prova de conceito. Para cada interface, serão avaliadas a inicialização, a validade dos argumentos, os códigos de retorno, o tratamento de *timeout*, a indicação explícita de operações não suportadas e a ausência de tipos específicos do SDK nas interfaces públicas. Quando um com-

portamento não puder ser exercitado diretamente em todas as placas, a limitação será registrada como ameaça à validade, sem ser tratada como conformidade plena.

O Quadro 6 apresenta as métricas e os respectivos procedimentos de coleta.

Quadro 6 – Métricas previstas para a validação

Métrica ou evidência	**Procedimento de coleta**	**Finalidade**
Cenários funcionais aprovados	Executar o mesmo roteiro nas duas versões das quatro placas e registrar saídas do *display*, LED e UART.	Verificar equivalência de comportamento.
Código compartilhado	Calcular $P_{comp,p}$ para cada plataforma e forma de implementação com as exclusões definidas.	Comparar o reúso efetivo de arquivos-fonte.
Arquivos comuns alterados	Comparar caminhos e hashes ao integrar cada plataforma.	Detectar mudanças distribuídas no núcleo comum.
Reúso do porte STM32F4	Comparar caminhos e hashes do porte nas construções STM32F411CEU6 e STM32F407VGT6.	Verificar o reúso integral dentro da família selecionada.
Código específico	Contar linhas do porte, BSP, inicialização e integração por plataforma.	Dimensionar a adaptação necessária.
Conformidade dos portes	Aplicar um roteiro comum de verificação dos contratos da HAL, cobrindo inicialização, argumentos inválidos, erros, *timeouts*, operações não suportadas, capacidades declaradas e ausência de tipos do SDK nas interfaces públicas.	Verificar se os portes apresentam comportamento compatível com o contrato comum dentro do recorte da prova de conceito.
Uso de *flash* e RAM	Registrar os relatórios de compilação das duas versões de cada placa sob configurações equivalentes.	Comparar de forma pareada o custo de memória.

Fonte: O Autor (2026).

Não serão estabelecidas antecipadamente metas numéricas de percentual compartilhado ou consumo de memória, pois não há medição preliminar que sustente limites arbitrários. Os critérios de aceitação serão: execução dos cenários nas quatro placas; identidade dos arquivos-fonte comuns; ausência de dependências de fabricante nesses arquivos; aprovação dos testes de conformidade; identidade do porte STM32CubeF4 nas duas placas STM32; e concentração das demais adaptações nos portes, BSPs, inicializações e arquivos de compilação. Caso um desses critérios não seja atendido, a arquitetura deverá ser revisada ou a limitação deverá ser explicitada nos resultados.

3.7 CRONOGRAMA DE IMPLEMENTAÇÃO

O desenvolvimento será distribuído ao longo de 14 semanas, conforme o Quadro 7. A sequência parte da estabilização dos contratos e dos componentes comuns, avança pelas plataformas e reserva as semanas finais para validação integrada, coleta das métricas e correções. Para reduzir diferenças acidentais de comportamento, cada implementação convencional será construída logo após a versão correspondente baseada no *framework*.

Quadro 7 – Cronograma de implementação em 14 semanas

Semana	**Atividade principal**
1	Criar os contratos de GPIO, I2C, UART e temporização e os mapeamentos das placas
2	Preparar os ambientes de desenvolvimento com cada um dos SDK.
3	Implementar o porte para ESP-IDF e o BSP da ESP32.
4	Implementar o porte STM32CubeF4 e os BSPs das placas STM32F411CEU6 e STM32F407VGT6.
5	Implementar o porte para nRF Connect SDK e o BSP da nRF52840.
6	Aplicar o roteiro comum de conformidade aos três portes e corrigir divergências em relação aos contratos.
7	Implementar e testar os componentes de botão, LED, temporização e telemetria sobre as interfaces comuns.
8	Implementar os componentes SHT31-D e SSD1306 e compor a aplicação comum.
9	Integrar a aplicação baseada no *framework* à ESP32, implementar a versão convencional para ESP-IDF e executar a verificação funcional pareada.
10	Integrar a aplicação baseada no *framework* à STM32F411CEU6, implementar a versão convencional correspondente e executar a verificação funcional pareada.
11	Integrar a STM32F407VGT6 nas duas formas de implementação, preservando o porte STM32CubeF4, e executar a verificação funcional pareada.
12	Integrar a aplicação baseada no *framework* à nRF52840, implementar a versão convencional para nRF Connect SDK e executar a verificação funcional pareada.
13	Executar os cenários funcionais completos e os casos de falha nas oito versões e coletar código compartilhado, código mantido, dispersão das dependências de plataforma, uso de *flash* e RAM.
14	Corrigir não conformidades, repetir as medições afetadas e consolidar a documentação e as evidências da implementação.

Fonte: O Autor (2026).